

ESPHome Workshop

สร้างอุปกรณ์ IoT ด้วย ESP32 โดยไม่ต้องเขียนโค้ด C++

เส้นทางจากเครื่องเปล่าสู่อุปกรณ์ IoT ที่ใช้งานได้จริงใน 3 ชั่วโมง



รูปที่ 1 ลำดับการทำงานตลอดหลักสูตร แต่ละช่วงจบด้วยผลลัพธ์ที่จับต้องได้

รายการ	รายละเอียด
รหัสหลักสูตร	IOT-101 · ไม่ต้องมีพื้นฐานเขียนโปรแกรม
ระยะเวลา	(180 นาที) ปฏิบัติ 90% บรรยาย 10%
ฮาร์ดแวร์	ESP32 DevKit V1 (ESP32-WROOM-32)
ซอฟต์แวร์อ้างอิง	ESPHome 2026.7 · Home Assistant 2026.x · Python 3.11+
เอกสาร	ฉบับย่อ v2.0 · สิงหาคม 2569

1 ผลลัพธ์และตารางเวลา

1.1 จบแล้วคุณจะทำอะไรได้

#	ความสามารถที่ได้	พิสูจน์ด้วย
1	ติดตั้งชุดเครื่องมือ ESPHome บนเครื่องตัวเอง	คำสั่ง esphome version ทำงานได้
2	เขียน YAML แล้วแฟลชลงบอร์ดจริงผ่าน USB	บอร์ดออนไลน์และเปิดหน้าเว็บของตัวเองได้
3	อ่านค่าเซนเซอร์ กรองสัญญาณ และคำนวณค่าใหม่	เห็นอุณหภูมิ ความชื้น และจุดน้ำค้าง
4	เชื่อมอุปกรณ์เข้า Home Assistant	อุปกรณ์ปรากฏบนหน้า Overview
5	อัปเดตเฟิร์มแวร์แบบไร้สาย และแก้ปัญหาจาก log เองได้	อัปเดตสำเร็จโดยไม่เสียบสาย USB

1.2 ตารางเวลา

เวลา	หัวข้อ	รูปแบบ
00:00 – 00:35	เปิดการอบรม ทฤษฎีที่จำเป็น (หัวข้อ 2–3)	บรรยาย + สาทิต
00:35 – 01:20	ติดตั้งเครื่องมือและเรียนรู้วิธีอัปเดต (หัวข้อ 4–5)	ปฏิบัติพร้อมกัน
01:20 – 01:45	Lab 1 บอร์ดตัวแรกออนไลน์	ปฏิบัติ
01:55 – 02:15	Lab 2 อ่านค่าเซนเซอร์ (พักเบรก 01:45 – 01:55)	ปฏิบัติ
02:15 – 02:40	Lab 3 Home Assistant และ OTA	ปฏิบัติ
02:40 – 03:00	Lab 4 และสรุป	ปฏิบัติ + สรุป

2 เช็คลิสต์ก่อนเริ่ม

2.1 อุปกรณ์ต่อผู้เรียนหนึ่งคน

✓	รายการ	จำนวน	ใช้ใน
☐	บอร์ด ESP32 DevKit V1 (30 ขา)	1	ทุก Lab
☐	สาย Micro-USB แบบรับส่งข้อมูลได้	1	ทุก Lab
☐	Breadboard 830 จุด และสายจัมเปอร์	1 ชุด	Lab 2–4
☐	เซนเซอร์ DHT22 (AM2302) และตัวต้านทาน 10 k Ω	อย่างละ 1	Lab 2
☐	LED 5 มม. + ตัวต้านทาน 220 Ω และปุ่มกด tactile	อย่างละ 1	Lab 4
☐	โมดูลรีเลย์ 1 ช่อง 5 V แบบ opto-isolated	1	Lab 4
☐	โน้ตบุ๊กที่ติดตั้งโปรแกรมได้ พื้นที่ว่างเกิน 5 GB	1	ทุก Lab

3 ทฤษฎีเท่าที่ต้งรู้

3.1 ESPHome ทำอะไร

ESPHome ไม่ใช่เฟิร์มแวร์สำเร็จรูป แต่เป็น **ตัวสร้างโค้ด** ที่แปลไฟล์ตั้งค่า YAML เป็นซอร์สโค้ด C++ แล้วคอมไพล์เป็นเฟิร์มแวร์เฉพาะอุปกรณ์นั้น ผลคือได้เฟิร์มแวร์ที่มีเฉพาะฟีเจอร์ที่เราประกาศ กินหน่วยความจำน้อย แต่เราเขียนแค่ไฟล์ตั้งค่าไม่กี่สิบบรรทัด



รูปที่ 2 ทุกครั้งที่แก้ไฟล์ YAML ต้องเดินครบทั้ง 5 ขั้นตอนนี้ใหม่เสมอ

คำศัพท์	ความหมาย	ตัวอย่าง
Component	ความสามารถหนึ่งอย่างที่เปิดใช้	wifi:, logger:, api:
Domain	ประเภทของสิ่งที่อุปกรณ์นำเสนอออกไป	sensor:, switch:, light:
Platform	การนำ domain ไปใช้กับฮาร์ดแวร์รุ่นหนึ่ง	platform: dht
ID	ชื่อภายในไฟล์ ใช้อ้างอิงกัน ไม่แสดงผลออกไป	id: temp_c
Lambda	โค้ด C++ สั้น ๆ แทรกในจุดที่ YAML ทำไม่ได้	return x * 2;

3.2 กฎ YAML 5 ข้อที่ทำให้พลาดบ่อยที่สุด

- ใช้เว้นวรรคเท่านั้น ห้ามกด Tab เด็ดขาด มาตรฐานคือย่อหน้าละ 2 เว้นวรรค
- ต้องมีเว้นวรรคหลังทวิภาค `pin: GPIO4` ถูก แต่ `pin:GPIO4` ผิด
- การเยื้องคือลำดับชั้น บรรทัดที่เยื้องลึกกว่าคือลูกของบรรทัดบน
- รายการขึ้นต้นด้วยขีดกลาง - `platform: dht`
- ตรวจสอบก่อนคอมไพล์เสมอ `esphome config` ไฟล์ `.yaml` ใช้เวลาไม่กี่วินาทีเพื่อจะได้ไม่เสียเวลาที่หลัง

3.3 key การตั้งค่า yaml

แม่แบบขั้นต่ำที่ใช้ได้จริง (ESPHome 2026.x)

```
esphome:
  name: esp-lab01                # ตัวพิมพ์เล็กและขีดกลางเท่านั้น
  friendly_name: "ESP Lab 01"
esp32:
  board: esp32dev                # เปลี่ยนตามบอร์ดที่ใช้
logger:                          # เปิดระบบ log
api:                             # ช่องทางคุยกับ Home Assistant
  encryption:
    key: !secret api_key_lab01   # บังคับตั้งแต่ 2026.1
ota:                             # เป็นรายการ ต้องมีขีดกลาง
  - platform: esphome
    password: !secret ota_password
wifi:
  ssid: !secret wifi_ssid
  password: !secret wifi_password
  ap:                            # จุดปล่อยสัญญาณสำรอง
    ssid: "ESP-Lab01 Fallback"
    password: !secret ap_password
captive_portal:
```

ตัวอย่างเก่าในอินเทอร์เน็ตจะคอมไพล์ไม่ผ่าน

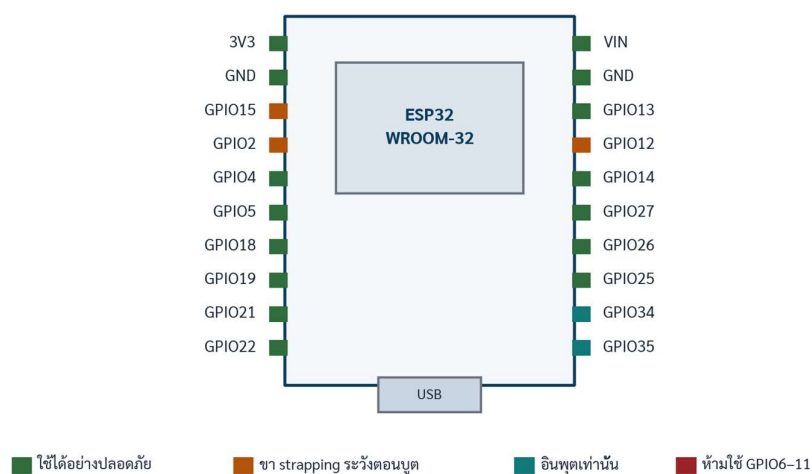
- ตั้งแต่ 2024.6 บล็อก `ota:` เป็นรายการ ต้องเขียน `- platform: esphome` นำหน้า
- ตั้งแต่ 2026.1 `api:` ยกเลิกรหัสผ่าน บังคับใช้ encryption key เท่านั้น
- ตั้งแต่ 2026.7 ESP32 ใช้ ESP-IDF เป็น toolchain เริ่มต้น ไฟล์เล็กลงและคอมไพล์เร็วขึ้น

3.4 วางตรรกะไว้ที่ไหน



รูปที่ 3 หลักง่าย ๆ คือ ตรรกะที่ห้ามล้มเหลว ต้องอยู่บนตัวอุปกรณ์เสมอ

3.5 เลือกขา GPIO ให้ถูก



รูปที่ 4 ขาที่ใช้บ่อยบน ESP32 DevKit V1 — เลือกจากกลุ่มสีเดียวกันก่อนเสมอ

สรุปเป็นข้อความ

- ปลอดภัย: 4, 5, 13, 14, 16–19, 21–23, 25–27, 32, 33
- อินพุตเท่านั้น ไม่มี pull-up ภายใน: 34, 35, 36, 39
- ระวังตอนบูต (strapping): 0, 2, 12, 15 · ห้ามใช้เด็ดขาด: 6–11
- ADC2 (0, 2, 4, 12–15, 25–27) อ่านค่าแอนะล็อกไม่ได้ขณะเปิด Wi-Fi

4 ติดตั้งเครื่องมือ

หัวข้อนี้ทำทีละข้อ **ห้ามข้าม** ถ้าข้อไหนยังไม่ผ่านให้แก้ไขให้ผ่านก่อนแล้วค่อยไปข้อถัดไป เมื่อจบหัวข้อนี้เครื่องของทุกคนต้องพิมพ์คำสั่ง esphome version แล้วมีเลขเวอร์ชันขึ้นมา

4.1 ต้องติดตั้งอะไรบ้าง

ลำดับ	สิ่งที่ติดตั้ง	ทำหน้าที่อะไร	เวลาโดยประมาณ
1	Python 3.11 ขึ้นไป	เป็นฐานให้โปรแกรม ESPHome ทำงานได้	5 นาที
2	ESPHome	ตัวแปลไฟล์ YAML เป็นเฟิร์มแวร์	5 นาที
3	ไดรเวอร์ USB-Serial	ทำให้คอมพิวเตอร์มองเห็นบอร์ด	5 นาที
4	VS Code	โปรแกรมสำหรับพิมพ์ไฟล์ YAML	5 นาที

4.2 รู้จักหน้าต่างคำสั่งก่อน

เราจะสั่งงานคอมพิวเตอร์ด้วยการพิมพ์คำสั่ง ไม่ใช่การคลิกไอคอน หน้าต่างนี้ Windows เรียกว่า Command Prompt ส่วน macOS กับ Linux เรียกว่า Terminal

1. **Windows** กดปุ่ม Windows บนคีย์บอร์ด พิมพ์ **cmd** แล้วกด Enter
2. **macOS** กด Command + Space พิมพ์ **Terminal** แล้วกด Enter
3. **Linux** กด Ctrl + Alt + T

กติกา	รายละเอียด
พิมพ์แล้วต้องกด Enter	คำสั่งจะยังไม่ทำงานจนกว่าจะกด Enter หนึ่งครั้ง
ตัวพิมพ์เล็กใหญ่ต่างกัน	esphome ใช้ได้ แต่ ESPHome ใช้ไม่ได้
วิธีวางข้อความ	Windows คลิกขวาหนึ่งครั้ง · macOS กด Command + V
บรรทัดที่ขึ้นต้นด้วย #	เป็นคำอธิบายของครู ไม่ต้องพิมพ์ตาม
หยุดโปรแกรมที่กำลังทำงาน	กด Ctrl + C หนึ่งครั้ง
เรียกคำสั่งเดิมกลับมา	กดปุ่มลูกศรขึ้น จะได้ไม่ต้องพิมพ์ซ้ำ

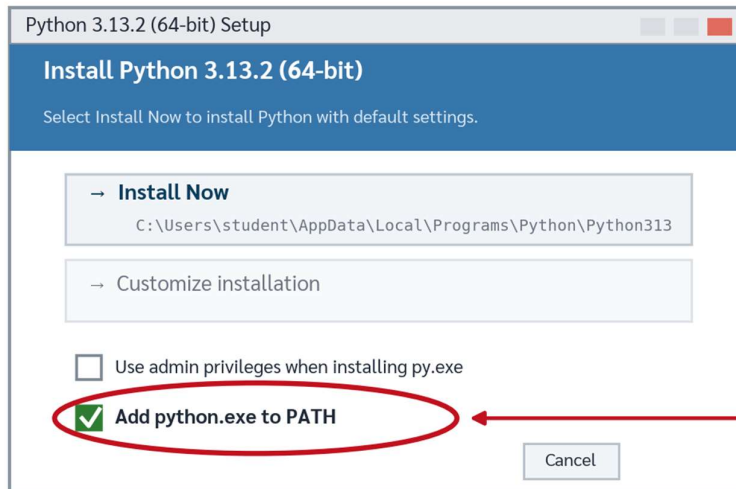
ห้ามพิมพ์ผิดแม้แต่ตัวเดียว

- คอมพิวเตอร์อ่านคำสั่งตรงตัวมาก **เว้นวรรคเกินหรือขาดหนึ่งช่อง** ก็ทำงานไม่ได้
- ถ้าพิมพ์ผิดแล้วขึ้น **ข้อความสีแดง** ไม่ต้องตกใจ ไม่มีอะไรพัง ให้อ่านข้อความแล้วพิมพ์ใหม่

4.3 ติดตั้ง Python บน Windows

1. เปิดเบราว์เซอร์ไปที่ python.org แล้วเลือกเมนู Downloads
2. กดปุ่มสี่เหลี่ยมที่เขียนว่า Download Python 3.x แล้วรอไฟล์โหลดเสร็จ
3. ดับเบิลคลิกไฟล์ที่โหลดมา จะมีหน้าต่างติดตั้งขึ้นมา
4. **ข้อสำคัญที่สุด** มองที่ด้านล่างของหน้าต่าง ให้ติ๊กช่อง Add python.exe to PATH ก่อน แล้วค่อยกด Install Now
5. รอประมาณ 2 นาที จนขึ้นข้อความ Setup was successful แล้วกด Close
6. ปิดหน้าต่างคำสั่งเดิมทั้งหมด แล้วเปิดใหม่ พิมพ์ `python --version` แล้วกด Enter

หน้าจอตัวติดตั้ง Python บน Windows



ต้องติ๊กช่องนี้ก่อน
ถ้าลืม จะพิมพ์คำสั่ง python
ในหน้าต่างคำสั่งไม่ได้

รูปที่ 5 จุดที่พลาดกันมากที่สุดคือช่องล่างสุด ต้องติ๊กก่อนกด Install Now

สิ่งที่ต้องเห็นบนหน้าจอ

```
C:\Users\student> python --version
Python 3.13.2
```

ต้องขึ้นเลขเวอร์ชันแบบนี้

ถ้าขึ้นข้อความว่า 'python' is not recognized

- แปลว่าลืมติ๊กช่อง Add python.exe to PATH ตอนติดตั้ง
- วิธีแก้: เปิดไฟล์ติดตั้งเดิมอีกครั้ง เลือก Modify แล้วกด Next จากนั้นติ๊ก Add Python to environment variables แล้วกด Install
- ติดตั้งเสร็จต้องปิดหน้าต่างคำสั่งแล้วเปิดใหม่เสมอ ไม่อย่างนั้นจะยังขึ้นข้อความเดิม

4.4 สร้างที่เก็บงานและกล่องเครื่องมือแยก

เราจะสร้างโฟลเดอร์เก็บงานหนึ่งโฟลเดอร์ แล้วสร้างสิ่งที่เรียกว่า **สภาพแวดล้อมเสมือน (venv)** ขึ้นใน คัดง่าย ๆ ว่าเป็นกล่องเครื่องมือเฉพาะของโครงการนี้ ที่ไม่ไปปนกับโปรแกรมอื่นในเครื่อง

Windows – พิมพ์ที่ละบรรทัด กด Enter ทุกบรรทัด

```
# บรรทัดที่ 1 สร้างโฟลเดอร์เก็บงาน
mkdir %USERPROFILE%\esphome-workshop

# บรรทัดที่ 2 เข้าไปในโฟลเดอร์นั้น
cd %USERPROFILE%\esphome-workshop

# บรรทัดที่ 3 สร้างกล่องเครื่องมือชื่อ venv (ใช้เวลาราว 20 วินาที)
python -m venv venv

# บรรทัดที่ 4 เปิดกล่องเครื่องมือขึ้นมาใช้
venv\Scripts\activate
```

สังเกตต้นบรรทัดหลังพิมพ์บรรทัดที่ 4

```
(venv) C:\Users\student\esphome-workshop>
^^^^ ต้องมีคำว่า (venv) อยู่หน้าสุด จึงจะถือว่าถูกต้อง
```

จำข้อนี้ให้ขึ้นใจ

- ทุกครั้งที่เปิดหน้าต่างคำสั่งใหม่ ต้องพิมพ์ `cd %USERPROFILE%\esphome-workshop` แล้วตามด้วย `venv\Scripts\activate` ซ้ำเสมอ
- ถ้าไม่เห็นคำว่า (venv) แล้วสั่งคำสั่ง esphome จะขึ้นว่าหาคำสั่งไม่พบ นี่คือสาเหตุที่พบบ่อยที่สุดในห้องเรียน

4.5 ติดตั้ง ESPHome

พิมพ์บรรทัดเดี๋ยวนี้อันแล้วรอ 2-5 นาที

```
pip install --upgrade pip esphome
```

ระหว่างรอจะเห็นข้อความวิ่งขึ้นมาเยอะมาก มีคำว่า Collecting และ Downloading ซ้ำ ๆ ถือเป็นเรื่องปกติ ห้ามปิดหน้าต่าง เมื่อเสร็จจะขึ้นบรรทัดที่ขึ้นต้นว่า Successfully installed

ตรวจสอบผลด้วยคำสั่งนี้

```
(venv) C:\...> esphome version
```

```
Version: 2026.7.1
```

เลขอาจต่างกันได้ ขอแค่มีเลขขึ้นมา

หน้าจอเมื่อติดตั้ง ESPHome สำเร็จ

```

Command Prompt

C:\Users\student>cd %USERPROFILE%\esphome-workshop

C:\Users\student\esphome-workshop>venv\Scripts\activate

(venv) C:\Users\student\esphome-workshop>esphome version
Version: 2026.7.1

(venv) C:\Users\student\esphome-workshop>_

```

ต้องมี (venv)
อยู่หน้าสุด

มีเลขเวอร์ชันขึ้นมา
= ติดตั้งสำเร็จแล้ว

รูปที่ 6 หน้าจอนี้คือเป้าหมายของหัวข้อ 4 ทั้งหมด ถ้าเห็นแบบนี้ถือว่าผ่าน

ข้อความที่ขึ้น	แปลว่า	วิธีแก้
'esphome' is not recognized	ยังไม่ได้เปิดกล่องเครื่องมือ	พิมพ์ venv\Scripts\activate แล้วลองใหม่
ERROR: Could not find a version	Python เก่าเกินไป	ติดตั้ง Python 3.11 ขึ้นไปตามหัวข้อ 4.3
Connection error / timed out	อินเทอร์เน็ตหลุดระหว่างโหลด	เช็ค Wi-Fi แล้วพิมพ์คำสั่งเดิมซ้ำอีกครั้ง
Access is denied	โฟลเดอร์ไม่มีสิทธิ์เขียน	ย้ายโฟลเดอร์งานไปไว้ที่ Documents แล้วเริ่มใหม่

4.6 สำหรับเครื่อง macOS และ Linux

macOS / Linux – Terminal

```
mkdir -p ~/esphome-workshop && cd ~/esphome-workshop
python3 -m venv venv
source venv/bin/activate          # ต้องเห็น (venv) ที่ต้นบรรทัด
pip install --upgrade pip esphome
esphome version
```

```
# Linux เท่านั้น: ขอสิทธิ์ใช้พอร์ต USB แล้วออกจากระบบและเข้าใหม่ 1 ครั้ง
sudo usermod -a -G dialout $USER
```

4.7 ติดตั้งไดรเวอร์ USB

บอร์ดกับคอมพิวเตอร์พูดกันคนละภาษา ไดรเวอร์คือตัวแปลภาษาให้ ถ้าไม่มีไดรเวอร์

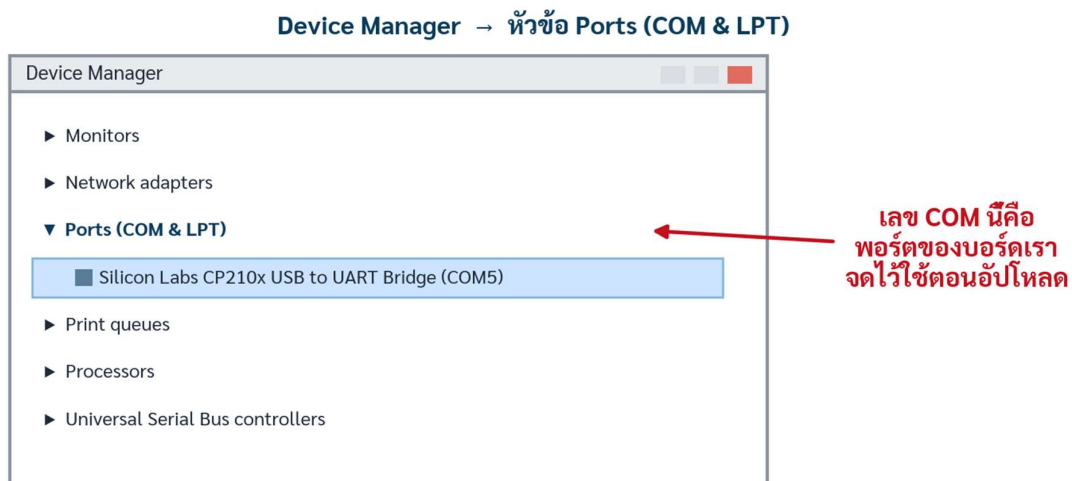
คอมพิวเตอร์จะไม่เห็นบอร์ดเลยแม้จะเสียบสายอยู่

1. พลิกดูชิปสี่เหลี่ยมเล็ก ๆ ที่อยู่ใกล้พอร์ต USB บนบอร์ด แล้วอ่านตัวอักษรบนชิป
2. เทียบกับตารางด้านล่างเพื่อดูว่าต้องโหลดไดรเวอร์ตัวไหน
3. ดาวน์โหลดจากเว็บไซต์ผู้ผลิต แยกไฟล์ แล้วติดตั้งตามปกติ
4. รีสตาร์ทคอมพิวเตอร์หนึ่งครั้งหลังติดตั้งเสร็จ

ตัวอักษรบนชิป	ชื่อไดรเวอร์	โหลดจาก
SILABS หรือ CP2102 / CP2104	CP210x VCP Driver	เว็บไซต์ Silicon Labs
WCH หรือ CH340 / CH9102	CH341SER	เว็บไซต์ WCH
ไม่มีชิปแยก มีพอร์ต USB สองช่อง	ไม่ต้องติดตั้ง	—

วิธีตรวจว่าคอมพิวเตอร์เห็นบอร์ดแล้วหรือยัง (Windows)

1. ยังไม่ต้องเสียบบอร์ด กดปุ่ม Windows + X แล้วเลือก Device Manager
2. หาหัวข้อ Ports (COM & LPT) แล้วกดสามเหลี่ยมเล็ก ๆ เพื่อกางออก จดชื่อที่มีอยู่เดิมไว้
3. เสียบสาย USB เข้ากับบอร์ด รายการใหม่จะโผล่ขึ้นมาเอง เช่น Silicon Labs CP210x (COM5)
4. เลข COM ที่เพิ่มขึ้นมาคือพอร์ตของบอร์ดเรา จดไว้ใช้ตอนอัปโหลด



รูปที่ 7 ถ้าไม่มีหัวข้อ Ports (COM & LPT) เลย แปลว่ายังไม่ได้ติดตั้งไดรเวอร์

สำหรับ macOS และ Linux

macOS - พิมพ์ก่อนและหลังเสียบสาย แล้วดูว่าอันไหนเพิ่มมา

```
ls /dev/cu.*
```

Linux

```
ls /dev/ttyUSB* /dev/ttyACM*
```

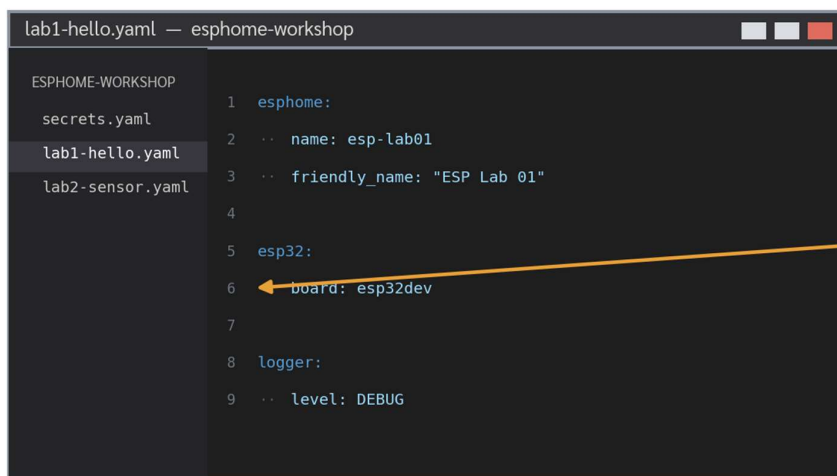
ไม่มีรายการใหม่โผล่ขึ้นมาเลย

- ให้เปลี่ยนสาย USB ก่อนเป็นอันดับแรก สายชาร์จโทรศัพท์จำนวนมากมีแต่เส้นไฟ ไม่มีเส้นส่งข้อมูล
- ลองเสียบช่อง USB อื่นของคอมพิวเตอร์ และหลีกเลี่ยงการต่อผ่านฮับ
- ถ้ายังไม่ขึ้น แปลว่ายังติดตั้งไดรเวอร์ไม่สำเร็จ ให้ติดตั้งใหม่แล้วรีสตาร์ทเครื่อง

4.8 โปรแกรมสำหรับพิมพ์ไฟล์ YAML

1. ดาวน์โหลดและติดตั้ง Visual Studio Code จาก code.visualstudio.com
2. เปิดโปรแกรม กดไอคอนรูปสี่เหลี่ยมสีขึ้นทางแถบซ้าย (Extensions) แล้วค้นคำว่า ESPHome จากนั้นกด Install
3. เปิดเมนู View แล้วเลือก Render Whitespace เพื่อให้มองเห็นช่องว่างเป็นจุดเล็ก ๆ
4. กด File แล้ว Open Folder เลือกโฟลเดอร์ esphome-workshop ที่สร้างไว้

หน้าจอ VS Code เมื่อเปิด Render Whitespace แล้ว



จุดเล็ก ๆ คือเว้นวรรค
ต้องมี 2 จุดต่อการเยื้อง
1 ระดับ ห้ามใช้ Tab

รูปที่ 8 เมื่อเปิด Render Whitespace แล้ว การเยื้องที่ผิดจะมองเห็นได้ทันที

ห้ามใช้ Microsoft Word พิมพ์ไฟล์ YAML

- Word จะแปลงเครื่องหมายคำพูดตรงให้เป็นคำพูดเอียงโดยอัตโนมัติ ทำให้ไฟล์ใช้ไม่ได้
- ถ้าไม่มี VS Code ให้ใช้ Notepad ได้ แต่ตอนบันทึกต้องเลือกชนิดไฟล์เป็น All Files และตั้งชื่อลงท้ายด้วย .yaml

4.9 สร้างโฟลเดอร์งานและไฟล์เก็บรหัสผ่าน

โครงสร้างโฟลเดอร์ที่จะได้เมื่อจบวันนี้

```

esphome-workshop/
├── venv/                กล้องเครื่องมือ ไม่ต้องแตะ
├── secrets.yaml         เก็บรหัสผ่านทั้งหมด
├── lab1-hello.yaml
├── lab2-sensor.yaml
├── lab3-ha.yaml
└── lab4-control.yaml

```

สร้างไฟล์ชื่อ `secrets.yaml` ใน VS Code แล้วพิมพ์ตามนี้ โดยเปลี่ยนค่าให้ตรงกับของห้องเรียน

```

secrets.yaml

wifi_ssid: "WORKSHOP-2G"      # ชื่อ Wi-Fi ห้องเรียน
wifi_password: "รหัสผ่านไวไฟ"
ap_password: "esphome1234"    # อย่างน้อย 8 ตัวอักษร
ota_password: "OtaW0rkshop2026"
api_key_lab01: "kR7v...="    # กุญแจยาว 44 ตัวอักษร

```

กุญแจในบรรทัดสุดท้ายสร้างได้จากหน้า Encryption Key Generator บนเว็บ esphome.io

กดปุ่มสร้างแล้วคัดลอกมาวางให้ครบทุกตัวอักษรรวมเครื่องหมาย `=` ตัวสุดท้าย

เช็คลิสต์ปิดท้ายหัวข้อ 4 — ต้องผ่านครบก่อนไปหัวข้อ 5

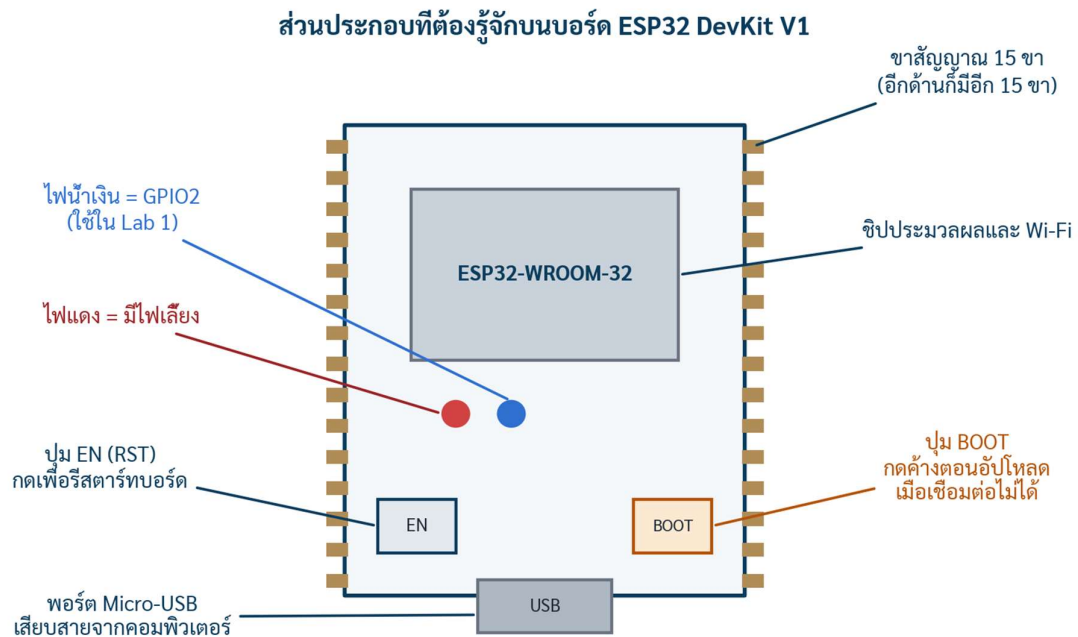
- ☐ พิมพ์ `python --version` แล้วขึ้นเลข 3.11 ขึ้นไป
- ☐ เห็นคำว่า `(venv)` ที่ต้นบรรทัดคำสั่ง
- ☐ พิมพ์ `esphome version` แล้วขึ้นเลขเวอร์ชัน
- ☐ เสียบบอร์ดแล้วเห็นพอร์ต COM ใหม่ใน Device Manager
- ☐ เปิดโฟลเดอร์ `esphome-workshop` ใน VS Code ได้
- ☐ มีไฟล์ `secrets.yaml` ที่กรอกค่าครบแล้ว

5 รู้จักบอร์ดและวิธีอัปโหลด

การอัปโหลดคือการส่งโปรแกรมจากคอมพิวเตอร์ **เข้าไปเก็บไว้ในบอร์ด** เมื่อส่งเสร็จแล้วบอร์ดจะทำงานเองได้ตลอด

แม้ถอดสายออกจากคอมพิวเตอร์

5.1 ส่วนประกอบบนบอร์ดที่ต้องรู้จัก



รูปที่ 9 ปุ่ม BOOT คือปุ่มที่จะได้ใช้บ่อยที่สุดเมื่ออัปโหลดไม่ผ่าน

5.2 เสียบบอร์ดครั้งแรก

1. จับบอร์ดที่ชอบ อย่าจับที่ขาโลหะโดยตรง
2. เสียบสาย Micro-USB เข้ากับพอร์ตบนบอร์ดให้แน่น แล้วเสียบอีกด้านเข้าคอมพิวเตอร์
3. **ต้องเห็นไฟสีแดงบนบอร์ดติดค้าง** ถ้าไม่ติดแปลว่าสายหรือพอร์ตมีปัญหา
4. เปิด Device Manager ตรวจสอบว่ามีพอร์ต COM ใหม่โผล่ขึ้นมา แล้วจดเลขพอร์ตไว้

ทำไมบางบอร์ดไฟน้ำเงินกะพริบตั้งแต่แรก

บอร์ดที่ซื้อใหม่มักมีโปรแกรมทดสอบจากโรงงานติดมาอยู่แล้ว เมื่อเราอัปโหลดโปรแกรมของเราลงไป โปรแกรมเดิมจะถูกเขียนทับทั้งหมด

5.3 การอัปโหลดหนึ่งครั้งมี 4 ขั้นตอน

สั่ง esphome run หนึ่งครั้ง ระบบจะทำครบทั้ง 4 ขั้นตอน



ข้อความตัวอย่างที่จะเห็นบนหน้าจอในแต่ละขั้น

รูปที่ 10 เราพิมพ์คำสั่งเดียว แต่เบื้องหลังระบบทำงานสี่ขั้นตอนตามลำดับ

5.4 ลงมืออัปโหลด

1. เปิดหน้าต่างคำสั่ง
2. เข้าโฟลเดอร์งาน `cd %USERPROFILE%\esphome-workshop`
3. เปิดกล่องเครื่องมือ `venv\Scripts\activate` แล้วต้องเห็น (venv)
4. ตรวจสอบไฟล์ก่อน `esphome config lab1-hello.yaml` ต้องไม่มีคำว่า ERROR
5. สั่งอัปโหลด `esphome run lab1-hello.yaml`
6. ระบบจะแสดงรายการพอร์ตให้เลือก ให้พิมพ์เลขข้อที่ตรงกับพอร์ต COM ของบอร์ด แล้วกด Enter
7. รอจนเสร็จ ห้ามถอดสายและห้ามปิดหน้าต่างระหว่างนี้

หน้าจอตอนให้เลือกพอร์ต

```
(venv) C:\...> esphome run lab1-hello.yaml
```

Please choose a device:

[1] COM5 (Silicon Labs CP210x USB to UART Bridge)

[2] Over The Air (esp-lab01.local)

(number): 1

<-- พิมพ์ 1 แล้วกด Enter

เลือกข้อไหนดี

- อัปโหลดครั้งแรกของบอร์ดใหม่ ต้องเลือกข้อที่เป็นพอร์ต COM หรือ USB เท่านั้น
- ตัวเลือก Over The Air จะใช้ได้ก็ต่อเมื่อบอร์ดเคยอัปโหลดผ่านสายมาแล้วอย่างน้อยหนึ่งครั้งและกำลังออนไลน์อยู่

5.5 ข้อความบนหน้าจอระหว่างอัปโหลด

ตัวอย่างข้อความจริงตั้งแต่ต้นจนจบ

```
INFO Reading configuration lab1-hello.yaml...
INFO Generating C++ source...
INFO Compiling app...
Compiling .pio\build\esp-lab01\src\main.cpp.o
... # ช่วงนี้ยาวที่สุด ให้รออย่างใจเย็น
Linking .pio\build\esp-lab01\firmware.elf
RAM: [==          ] 15.2% (used 49876 bytes)
Flash: [=====   ] 54.1% (used 709234 bytes)
===== [SUCCESS] =====

INFO Uploading ../../firmware.bin (742096 bytes)
Connecting.... # กำลังตรงนี้ให้กดปุ่ม BOOT
Writing at 0x00010000... (12 %)
Writing at 0x0001f000... (100 %)
Hash of data verified.
Leaving... Hard resetting via RTS pin...

INFO Successfully uploaded program.
[C][wifi:xxx]: IP Address: 192.168.1.87 <-- จดเลขนี้ไว้
```

หน้าจอตอนอัปโหลดสำเร็จ

```
Command Prompt — esphome run

===== [SUCCESS] =====

INFO Uploading .pioenvs/esp-lab01/firmware.bin (742096 bytes)
Connecting....
Chip is ESP32-D0WD-V3 (revision v3.1)
Writing at 0x00010000... (12 %)
Writing at 0x0001f4a0... (67 %)
Writing at 0x0002c118... (100 %)
Hash of data verified.
Leaving... Hard resetting via RTS pin...

INFO Successfully uploaded program.
[C][wifi:600]: IP Address: 192.168.1.87
```

ค้างตรงนี้นานเกิน 10 วินาที
ให้กดปุ่ม BOOT ค้างไว้

เห็นสองบรรทัดนี้
= สำเร็จแล้ว จด IP ไว้

รูปที่ 11 สองบรรทัดสุดท้ายคือสิ่งที่ต้องเห็น ถ้าเห็นแล้วแปลว่าโปรแกรมอยู่ในบอร์ดเรียบร้อยแล้ว

คำที่เห็น	แปลว่า
Compiling	กำลังแปลงโปรแกรม ใช้เวลานานที่สุด รอได้
[SUCCESS]	แปลงโปรแกรมเสร็จสมบูรณ์แล้ว
Connecting....	กำลังคุยกับบอร์ด ถ้าจูดึงยาวเกิน 10 วินาที ให้กดปุ่ม BOOT
Writing at ... (%)	กำลังเขียนลงบอร์ดจริง ห้ามถอดสายตอนนี้เด็ดขาด
Hash of data verified	ตรวจแล้วข้อมูลถูกต้องครบถ้วน
Successfully uploaded program	อัปโหลดสำเร็จ บอร์ดกำลังเริ่มทำงาน

5.6 ปุ่ม BOOT ใช้ตอนไหนและกดอย่างไร

บอร์ดบางรุ่นไม่ยอมเข้าโหมดรับโปรแกรมเอง ต้องช่วยด้วยการกดปุ่มค้างไว้ จังหวะการกดสำคัญมาก

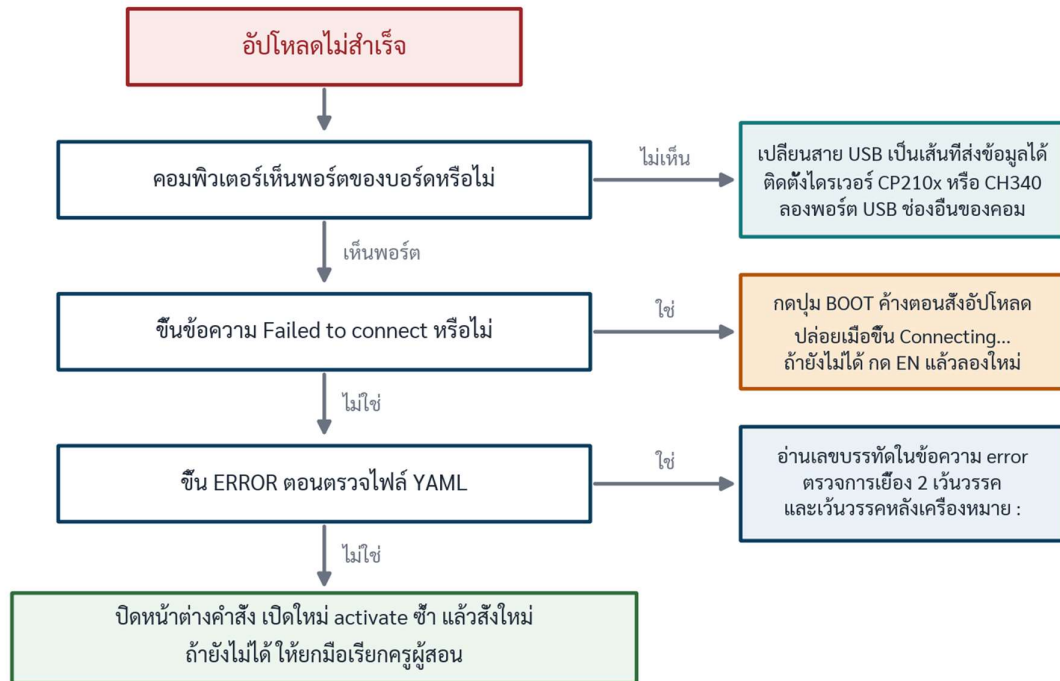
- พิมพ์คำสั่ง `esphome run` แล้วกด Enter ตามปกติ
- รอจนเห็นคำว่า Connecting.... ขึ้นบนหน้าจอ
- กดปุ่ม BOOT ค้างไว้ นับ 1 ถึง 3 ในใจ
- เมื่อเห็นคำว่า Writing at ปรากฏขึ้นแล้ว จึงปล่อยปุ่มได้
- ถ้ายังไม่สำเร็จ ให้กดปุ่ม EN หนึ่งครั้งเพื่อรีเซ็ตบอร์ด แล้วเริ่มใหม่ตั้งแต่ข้อ 1

5.7 รู้ได้อย่างไรว่าอัปโหลดสำเร็จ

ครบสี่ข้อนี้ถือว่าสำเร็จ

- ☐ เห็นข้อความ INFO Successfully uploaded program
- ☐ หลังจากนั้นข้อความ log วิ่งขึ้นมาต่อเนื่อง
- ☐ ในข้อความ log มีบรรทัด IP Address พร้อมตัวเลข
- ☐ นำเลข IP ไปเปิดในเบราว์เซอร์แล้วเห็นหน้าเว็บของบอร์ด

5.8 ถ้าอัปโหลดไม่สำเร็จให้ทำตามดังนี้



รูปที่ 12 ไต่จากบนลงล่างทีละข้อ อย่าข้ามขั้น

5.9 ข้อความผิดพลาดที่พบบ่อยตอนอัปโหลด

ข้อความที่ขึ้น	สาเหตุ	วิธีแก้
Failed to connect to ESP32	บอร์ดไม่เข้าโหมดรับโปรแกรม	กดปุ่ม BOOT ค้างตามหัวข้อ 5.6
No serial port found	คอมพิวเตอร์ไม่เห็นบอร์ด	เปลี่ยนสาย USB และตรวจไดรเวอร์ตามหัวข้อ 4.7
Serial port is busy	มีอีกหน้าต่างเปิด log ค้างอยู่	กด Ctrl + C ในหน้าต่างเดิม หรือปิดหน้าต่างนั้นทิ้ง

ข้อความที่ขึ้น	สาเหตุ	วิธีแก้
A fatal error occurred	สายหลวมหรือไฟไม่พอ	เสียบสายให้แน่น เปลี่ยนช่อง USB ไม่ต่อผ่านฮับ
could not determine a constructor	ไฟล์ YAML เยื้องผิดหรือใช้ Tab	เปิด Render Whitespace แล้วแก้ไขเป็นเว้นวรรค 2 ช่อง
Unable to connect to esp- xxx.local	หาบอร์ดในเครือข่ายไม่เจอ	ใช้ --device ตามด้วยเลข IP แทนชื่อ

5.10 การหยุดดู log และกลับมาดูใหม่

- หยุดดู log ให้กด **Ctrl + C** หนึ่งครั้ง โปรแกรมในบอร์ดยังทำงานต่อตามปกติ ไม่ได้ถูกลบ
- อยากกลับมาดู log อีกครั้งโดยไม่อัปเดตใหม่ ให้พิมพ์ `esphome logs lab1-hello.yaml`
- ถ้าอยากเริ่มคอมไพล์ใหม่ทั้งหมดเพราะมีการแปลก ๆ ให้พิมพ์ `esphome clean lab1-hello.yaml` ก่อน
แล้วค่อยสั่ง run ใหม่

5.11 ครั้งต่อไปไม่ต้องใช้สายแล้ว

เมื่อบอร์ดอัปเดตผ่านสายสำเร็จหนึ่งครั้งและต่อ Wi-Fi ได้แล้ว ครั้งถัดไปเลือกตัวเลือก Over The Air ได้เลย

ระบบจะส่งโปรแกรมผ่าน Wi-Fi ให้ฝึกทำจริงในปฏิบัติการที่ 3

LAB 1 บอร์ดตัวแรกออนไลน์

25 นาที · ไฟล์ lab1-hello.yaml · ไม่ต้องต่อวงจรเพิ่ม

ผลลัพธ์ที่ต้องได้

บอร์ดต่อ Wi-Fi ได้ มีหน้าเว็บของตัวเอง และกดเปิดปิดไฟ LED บนบอร์ดจากหน้าเว็บได้

ขั้นตอน

1. เข้าโฟลเดอร์โครงการและ activate สภาพแวดล้อมเสมือน จนเห็น `(venv)`
2. สร้างไฟล์ `lab1-hello.yaml` ตามโค้ดด้านล่าง
3. ตรวจสอบไวยากรณ์ `esphome config lab1-hello.yaml` ต้องไม่มี ERROR
4. เสียบบอร์ดเข้าคอมพิวเตอร์
5. สั่ง `esphome run lab1-hello.yaml` แล้วเลือกพอร์ต USB ของบอร์ด
6. รอคอมไพล์ (ครั้งแรก 5-15 นาที) ระบบจะอัปเดตและเปิด log ให้อัตโนมัติ
7. หา IP ของบอร์ดจากบรรทัด `IP Address` ใน log แล้วเปิดในเบราว์เซอร์

อัปเดตไม่ผ่าน ขึ้น Failed to connect

กดปุ่ม BOOT บนบอร์ดค้างไว้ตอนเริ่มอัปเดต แล้วปล่อยเมื่อขึ้นข้อความ Connecting

ไฟล์ lab1-hello.yaml

```
substitutions:
  device_name: esp-lab01

esphome:
  name: ${device_name}
  friendly_name: "ESP Lab 01"
esp32:
  board: esp32dev
logger:
  level: DEBUG
api:
  encryption:
    key: !secret api_key_lab01
ota:
  - platform: esphome
    password: !secret ota_password
wifi:
  ssid: !secret wifi_ssid
  password: !secret wifi_password
  ap:
    ssid: "ESP-Lab01 Fallback"
    password: !secret ap_password
captive_portal:

web_server:
  port: 80                                # หน้าเว็บควบคุมในตัวอุปกรณ์

output:
  - platform: gpio
    pin: GPIO2                            # LED บนบอร์ด ต่อมาแล้วจากโรงงาน
    id: onboard_led_out

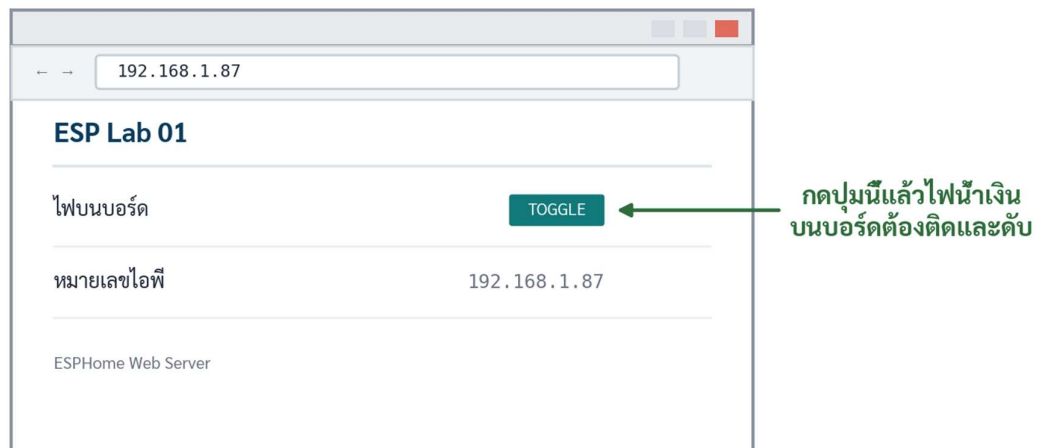
light:
  - platform: binary
    name: "ไฟบนบอร์ด"
    id: onboard_led
    output: onboard_led_out

text_sensor:
  - platform: wifi_info
    ip_address:
      name: "หมายเลขไอพี"
```

อ่าน log ให้เป็น

อักขรนำหน้า	ระดับ	ใช้คืออะไร
[E]	ERROR	ข้อผิดพลาดร้ายแรง ต้องแก้
[W]	WARNING	ผิดปกติแต่ไปต่อได้ เช่น Wi-Fi หลุดชั่วคราว
[C]	CONFIG	สรุปค่าตั้งจริงตอนบูต มีบรรทัด IP Address อยู่ตรงนี้
[D]	DEBUG	การทำงานที่ละเอียดขึ้น เช่น การอ่านค่าเซนเซอร์

ผลลัพธ์ที่ต้องเห็นเมื่อจบ Lab 1



รูปที่ 13 หน้าเว็บของบอร์ดจะหน้าตาแบบนี้ กดปุ่มแล้วไฟบนบอร์ดต้องเปลี่ยนทันที

จุดตรวจสอบ Lab 1

- ☐ esphome run จบโดยไม่มี ERROR
- ☐ ใน log พบบรรทัด IP Address
- ☐ เปิดเบราว์เซอร์ไปที่ IP นั้นแล้วเห็นหน้าเว็บของอุปกรณ์
- ☐ กดปุ่มบนหน้าเว็บแล้วไฟ LED สีน้ำเงินติดและดับตามได้จริง

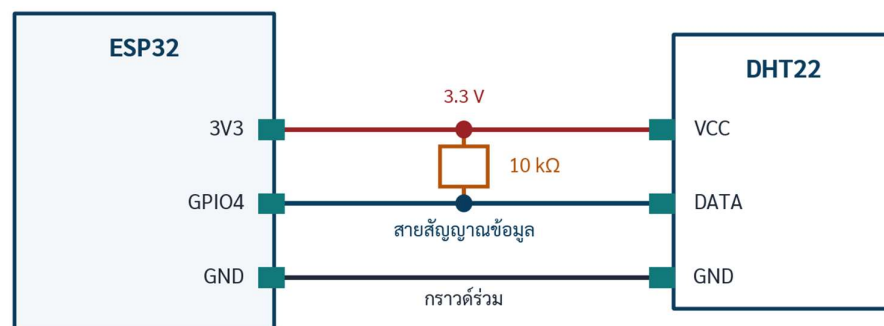
LAB 2 อ่านค่าเซนเซอร์และคำนวณค่าใหม่

20 นาที · ไฟล์ lab2-sensor.yaml · ใช้ DHT22

ผลลัพธ์ที่ต้องได้

หน้าเว็บของบอร์ดแสดงอุณหภูมิ ความชื้น และจุดน้ำค้างที่คำนวณเอง พร้อมส่งไฟติดอัตโนมัติเมื่อร้อนเกินเกณฑ์

ต่อวงจร



หากใช้โมดูล DHT22 แบบมีแผ่นวงจร 3 ขา มักมีตัวต้านทาน pull-up มาให้แล้ว ไม่ต้องเพิ่ม

รูปที่ 14 ถอดสาย USB ก่อนต่อวงจรเสมอ และอย่าลืมตัวต้านทาน pull-up

ขั้นตอน

1. คัดลอก `lab1-hello.yaml` เป็น `lab2-sensor.yaml` แล้วเปลี่ยน `device_name` เป็น `esp-lab02`
2. ต่อวงจรตามรูป โดยยังไม่เสียบสาย USB
3. เพิ่มโค้ดด้านล่างต่อท้ายไฟล์
4. เสียบสาย USB แล้วสั่ง `esphome run lab2-sensor.yaml`
5. เปิดหน้าเว็บของบอร์ด แล้วใช้มือกำเซนเซอร์ไว้ 30 วินาที สังเกตค่าความชื้นเพิ่มขึ้น

โค้ดที่เพิ่ม

```

sensor:
- platform: dht
  pin: GPIO4
  model: DHT22
  update_interval: 10s          # ห้ามต่ำกว่า 2s
  temperature:
    name: "อุณหภูมิ"
    id: temp_c
    accuracy_decimals: 1
    filters:
      - filter_out: nan          # ทิ้งค่าที่อ่านไม่ได้
      - sliding_window_moving_average:
          window_size: 5
          send_every: 1          # ส่งค่าเฉลี่ยใหม่ทุกครั้งที่อ่าน
    on_value_range:              # อัปเดตโมดูลที่ทำงานบนตัวบอร์ด
      - above: 30.0
        then:
          - light.turn_on: onboard_led
      - below: 29.0
        then:
          - light.turn_off: onboard_led
  humidity:
    name: "ความชื้นสัมพัทธ์"
    id: humi
    accuracy_decimals: 0
    filters:
      - filter_out: nan

- platform: template              # เซนเซอร์ที่คำนวณเอง
  name: "จุดน้ำค้าง"
  unit_of_measurement: "°C"
  device_class: temperature
  accuracy_decimals: 1
  update_interval: 10s
  lambda: |-
    float tc = id(temp_c).state;
    float rh = id(humi).state;
    if (isnan(tc) || isnan(rh) || rh <= 0) return {};
    const float a = 17.62, bb = 243.12;
    float g = log(rh / 100.0) + (a * tc) / (bb + tc);
    return (bb * g) / (a - g);

- platform: wifi_signal
  name: "ความแรงสัญญาณ"
  update_interval: 60s

binary_sensor:
- platform: status
  name: "สถานะออนไลน์"

```

ตัวกรองที่ใช้บ่อย

ตัวกรอง	หน้าที่	ใช้เมื่อ
filter_out: nan	ทิ้งค่าที่อ่านผิดพลาด	เซนเซอร์ตระกูล DHT
sliding_window_moving_average	เฉลี่ยแบบหน้าต่างเลื่อน	ค่าที่แกว่งไปมา
offset / multiply	ขจัดเซย์หรือคูณสเกล	สอบเทียบกับเครื่องมือมาตรฐาน
delta	ส่งค่าเฉพาะเมื่อเปลี่ยนเกินที่กำหนด	ลดข้อมูลที่ส่งขึ้นเครือข่าย
lambda	เขียนสูตรคำนวณเอง	แปลงหน่วยหรือสมการเฉพาะทาง

จุดตรวจสอบ Lab 2

- ☐ หน้าเว็บแสดงอุณหภูมิและความชื้นเป็นตัวเลขที่สมเหตุสมผล ไม่ใช่ค่าว่าง
- ☐ ค่าจุดน้ำค้างต่ำกว่าอุณหภูมิห้องเสมอ
- ☐ ถ้าเซนเซอร์ไว้แล้วค่าความชื้นเพิ่มขึ้นชัดเจน
- ☐ เมื่ออุณหภูมิเกิน 30 องศา ไฟ LED ติดขึ้นเองโดยไม่ต้องสั่ง

LAB 3 เข้า Home Assistant และอัปเดตไร้สาย

25 นาที · ไฟล์ lab3-ha.yaml

ผลลัพธ์ที่ต้องได้

อุปกรณ์ปรากฏใน Home Assistant พร้อมทุกรายการ และอัปเดตเฟิร์มแวร์ใหม่ได้โดยไม่ต้องเสียบสาย USB

ส่วนที่ 1 — เพิ่มอุปกรณ์เข้า Home Assistant

1. ตรวจสอบว่าอุปกรณ์และ Home Assistant อยู่ในเครือข่ายวงเดียวกัน
2. เข้า Settings แล้วเลือก Devices & Services
3. ปกติระบบจะค้นพบอุปกรณ์เองและขึ้นการ์ดแจ้งเตือน กด Configure ได้ทันที
4. ถ้าไม่พบ กด Add Integration เลือก ESPHome แล้วกรอก IP ของอุปกรณ์ พอร์ต 6053
5. วาง Encryption Key ให้ตรงกับค่าในคีย์ `api_key_lab01` ของ secrets.yaml
6. กำหนดพื้นที่ (Area) แล้วกด Finish จากนั้นดูรายการทั้งหมดบนหน้า Overview

Home Assistant มองไม่เห็นอุปกรณ์

- สาเหตุอันดับหนึ่งคือ mDNS ถูกบล็อกหรือข้ามเครือข่ายย่อยไม่ได้ ให้เพิ่มด้วย IP โดยตรง
- ถ้าขึ้นว่าถูกจำเริญถูกต้อง ให้ตรวจว่าคัดลอกครบ 44 ตัวอักษรรวมเครื่องหมายเท่ากับท้ายสุด
- ตรวจสอบ IP ให้อุปกรณ์ที่เราเตอร์ (DHCP reservation) กัน IP เปลี่ยนหลังไฟดับ

ส่วนที่ 2 — เพิ่มความสามารถฟังก์ชัน

```
time:
  - platform: homeassistant      # ชิงค์เวลาจากเซิร์ฟเวอร์
    id: ha_time

button:
  - platform: restart
    name: "สั่งรีเซ็ตอุปกรณ์"
```

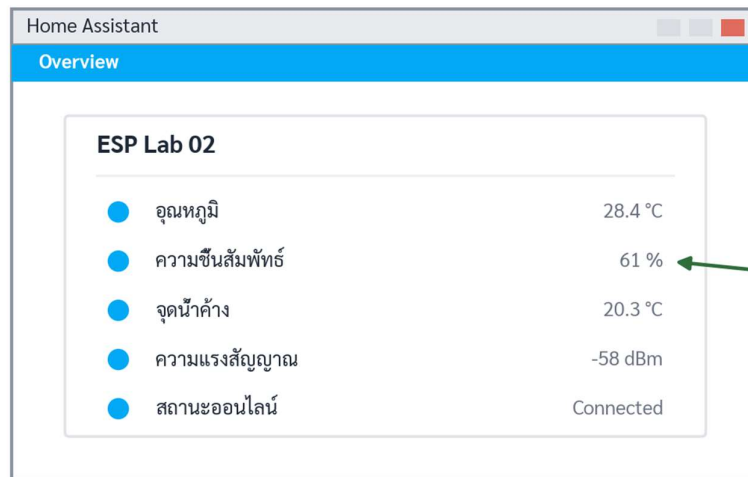
ส่วนที่ 3 — อัปเดตแบบไร้สาย (OTA)

1. ถอดสาย USB แล้วจ่ายไฟบอร์ดด้วยอะแดปเตอร์หรือพาวเวอร์แบงก์
2. แกะไฟล์เล็กน้อย เช่น เปลี่ยนชื่อรายการอุณหภูมิ
3. สั่ง `esphome run lab3-ha.yaml` อีกครั้ง
4. เลือกปลายทางที่เป็น IP หรือชื่อลงท้าย .local แทนพอร์ต USB
5. หรือระบุตรง ๆ ด้วย `esphome run lab3-ha.yaml --device 192.168.1.87`
6. เมื่ออัปเดตครบ 100% อุปกรณ์จะรีบูตและกลับมาออนไลน์เองในไม่กี่วินาที

ข้อจำกัดของ OTA ที่ต้องรู้

- บอร์ดใหม่ต้องแฟลชผ่าน USB ครั้งแรกเสมอ OTA ใช้ได้ตั้งแต่ครั้งที่สอง
- ถ้าเฟิร์มแวร์ใหม่ต่อ Wi-Fi ไม่ได้ อุปกรณ์จะเข้า safe mode และเปิดจุดปล่อยสัญญาณสำรองให้แก้ไข
- อุปกรณ์ที่รัน 2026.1 ขึ้นไปต้องยืนยันตัวตนแบบ SHA256 จึงอัปเดตจากเครื่องที่ติดตั้ง ESPHome เก่ากว่า 2025.10 ไม่ได้

ผลลัพธ์ที่ต้องเห็นเมื่อจบ Lab 3



← ค่าต้องเปลี่ยนตามจริง
ภายในไม่กี่วินาที

รูปที่ 15 เมื่อเพิ่มอุปกรณ์สำเร็จ รายการทั้งหมดจะขึ้นมาเองแบบนี้

จุดตรวจสอบ Lab 3

- ☐ อุปกรณ์ปรากฏใน Home Assistant พร้อมรายการครบ
- ☐ ค่าอุณหภูมิบนหน้า Overview เปลี่ยนตามค่าจริงภายในไม่กี่วินาที
- ☐ กดควบคุมไฟ LED จาก Home Assistant ได้
- ☐ อัปเดตเฟิร์มแวร์ใหม่สำเร็จโดยไม่ได้เสียบสาย USB

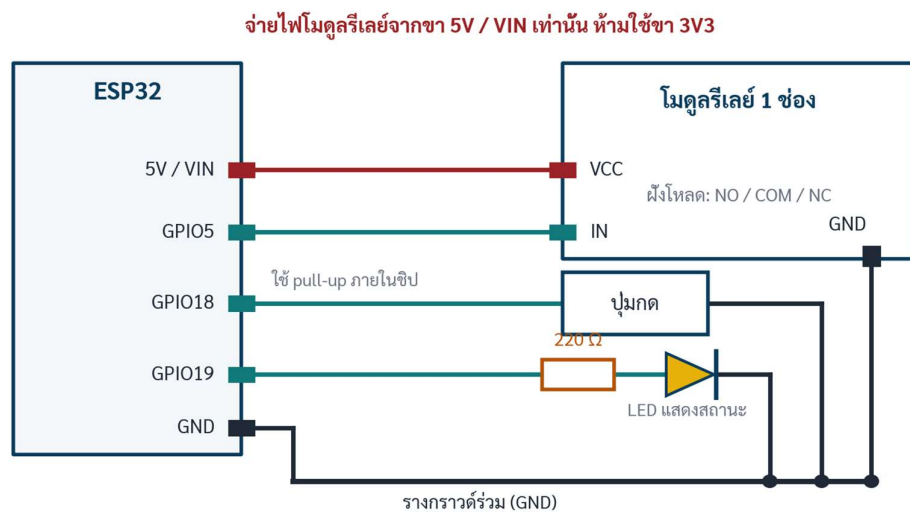
LAB 4 สวิตช์อัจฉริยะที่ทำงานได้แม้เน็ตหลุด

20 นาที · ไฟล์ lab4-control.yaml · ระดับท้าทาย

ผลลัพธ์ที่ต้องได้

- กดปุ่มสวิตช์เพื่อสลับสถานะรีเลย์ กดค้างเกิน 2 วินาทีเพื่อบังคับปิด
- ไฟสถานะภายนอกติดตามการทำงานของรีเลย์
- ตัวรีเลย์อัตโนมัติเมื่อทำงานครบ 30 นาที และคืนสถานะเป็นปิดเสมอเมื่อไฟดับแล้วกลับมา

ต่อวงจร



รูปที่ 16 ทดสอบด้วยเสียงคลิกของรีเลย์และไฟสถานะเท่านั้น ห้ามต่อไฟ 220 โวลต์

โค้ดที่เพิ่ม

```
switch:
- platform: gpio
  pin: GPIO5
  id: relay_1
  name: "รีเลย์ควบคุมหลอด"
  restore_mode: ALWAYS_OFF      # ไฟกลับมาแล้วต้องปิดไว้ก่อน
  on_turn_on:
    - light.turn_on: status_lamp
    - delay: 30min               # ตัวจับเวลาความปลอดภัย
    - switch.turn_off: relay_1
  on_turn_off:
    - light.turn_off: status_lamp

output:
- platform: gpio
  pin: GPIO19
  id: status_led_out

light:
- platform: binary
  name: "ไฟแสดงสถานะ"
  id: status_lamp
  output: status_led_out

binary_sensor:
- platform: gpio
  name: "ปุ่มกดหน้าเครื่อง"
  pin:
    number: GPIO18
    mode:
      input: true
      pullup: true
      inverted: true             # กดแล้วให้เป็นสถานะ true
  filters:
    - delayed_on: 30ms          # กันสัญญาณค้างของหน้าสัมผัส
  on_click:
    - min_length: 50ms          # กดสั้น = สลับสถานะ
      max_length: 800ms
      then:
        - switch.toggle: relay_1
    - min_length: 2s             # กดค้าง = บังคับปิด
      max_length: 6s
      then:
        - switch.turn_off: relay_1
```

- logger.log: "ผู้ใช้งานปิดฉุกเฉิน"

ทำไมต้องเขียนแบบนี้

ส่วนของโค้ด	เหตุผลเชิงวิศวกรรม
restore_mode: ALWAYS_OFF	อุปกรณ์ควบคุมไหลจะต้องอยู่ในสถานะปลอดภัยเมื่อไฟกลับมา
delay ใน on_turn_on	ตัวจับเวลาบนตัวอุปกรณ์ ถ้าสั่งเปิดใหม่ระหว่างรอ จะเริ่มนับใหม่
delayed_on: 30ms	กรองสัญญาณเด็งของปุ่ม ต้นเหตุอันดับหนึ่งของการสั่งซ้ำ
on_click พร้อมช่วงเวลา	ปุ่มเดียวทำสองหน้าที่โดยไม่ต้องเพิ่มฮาร์ดแวร์

จุดตรวจสอบ Lab 4

- ☐ กดสั้นแล้วได้ยินเสียงคลิกกรีย์และไฟสถานะติด
- ☐ กดค้างเกิน 2 วินาทีแล้วรีเลย์ปิดทันที และพบข้อความใน log
- ☐ ปิดเรเตอร์แล้วปุ่มยังทำงานได้ตามปกติ
- ☐ ตัดไฟแล้วจ่ายใหม่ รีเลย์ต้องอยู่ในสถานะปิด

10 ตารางแก้ปัญหาฉบับรวม

อาการ	สาเหตุ	วิธีแก้
ไม่พบพอร์ตอนุกรม	ไดรเวอร์ไม่ครบ หรือสายไม่มีเส้นข้อมูล	ติดตั้งไดรเวอร์ตามหัวข้อ 4.7 และเปลี่ยนสาย

อาการ	สาเหตุ	วิธีแก้
Failed to connect to ESP32	บอร์ดไม่เข้าโหมดรับโปรแกรม	กดปุ่ม BOOT ค้างตามหัวข้อ 5.6
YAML error เรื่องโครงสร้าง	เยื้องผิดหรือใช้ Tab	ใช้เว้นวรรค 2 ช่อง ตรวจสอบด้วย esphome config
Wi-Fi ต่อบ้างไม่จบ	เครือข่ายเป็น 5 GHz หรือรหัสผ่านผิด	ใช้ย่าน 2.4 GHz และตรวจสอบ secrets.yaml
บอร์ดรีบูตซ้ำทุกไม่กี่วินาที	กระแสไม่พอ มักเกิดตอนจ่ายรีเลย์จาก 3V3	ใช้แหล่งจ่าย 5V 1A ขึ้นไป และจ่ายโหนดจาก VIN
เซนเซอร์อ่านได้เป็น nan	ไม่มี pull-up หรืออ่านถี่เกินไป	ใส่ 10 k Ω และตั้ง update_interval อย่างน้อย 2s
HA มองไม่เห็นอุปกรณ์	mDNS ถูกบล็อก	เพิ่มด้วย IP โดยตรงที่พอร์ต 6053
ขึ้นว่ากฎไม่ถูกต้อง	คัดลอกกฎไม่ครบ	ลบ integration เดิมแล้วเพิ่มใหม่
OTA ล้มเหลวกลางคัน	สัญญาณอ่อนหรือเวอร์ชันเครื่องมือเก่า	เข้าใกล้จุดกระจายสัญญาณ และอัปเดต ESPHome
ไม่มี log ปรากฏเลย	เลือกพอร์ตผิด หรือบอร์ด S3 ยังไม่ตั้ง USB CDC	เลือกพอร์ตให้ถูก และตรวจสอบการตั้งค่า logger
ค่าอ่านได้แต่แกว่งมาก	สัญญาณรบกวนหรือสายยาวเกิน	เพิ่ม sliding_window_moving_average

11 สรุปและก้าวต่อไป

11.1 คำสั่งที่ต้องจำ

คำสั่ง	หน้าที่
esphome config ไฟล์.yaml	ตรวจไวยากรณ์โดยไม่คอมไพล์ — ใช้ทุกครั้งก่อน run
esphome run ไฟล์.yaml	ตรวจ คอมไพล์ อัปเดต และเปิด log ในคำสั่งเดียว
esphome logs ไฟล์.yaml	ดู log อย่างเดียว ไม่คอมไพล์ใหม่
esphome clean ไฟล์.yaml	ล้างไฟล์ที่สร้างไว้ ใช้เมื่อคอมไพล์ผิดปกติ
esphome run ไฟล์.yaml --device <IP>	ระบุปลายทางตรง ๆ ใช้เมื่อ mDNS ใช้ไม่ได้
pip install --upgrade esphome	อัปเดตเป็นเวอร์ชันล่าสุด

11.2 แนวปฏิบัติเมื่อนำไปใช้จริง

- ตั้งชื่ออุปกรณ์ตามตำแหน่งติดตั้งจริง เช่น sensor-lab-a-01 ไม่ใช่ esp32-1
- เก็บไฟล์ตั้งค่าใน Git แต่ไม่รวม secrets.yaml และสำรองกุญแจเข้ารหัสไว้เสมอ
- ตั้งเวอร์ชัน ESPHome ในงานที่ส่งมอบลูกค้า และอ่านหมายเหตุก่อนอัปเดตทุกครั้ง
- ใส่ sensor แบบ uptime และ wifi_signal ในทุกอุปกรณ์ เพื่อเฝ้าระวังสุขภาพระบบ
- ตรวจระดับความปลอดภัยต้องอยู่บนตัวอุปกรณ์เสมอ ไม่ฝากไว้กับเซิร์ฟเวอร์
- แยก VLAN สำหรับอุปกรณ์ IoT และห้ามเปิดพอร์ตออกอินเทอร์เน็ตโดยตรง

แหล่งอ้างอิงที่ควรBookmark

- esphome.io — เอกสารทางการ ค้นหาคอมโพเนนต์และตัวเลือกทั้งหมด
- esphome.io/changelog — อ่านก่อนอัปเดตเวอร์ชันทุกครั้ง
- github.com/esphome/esphome และ community.home-assistant.io — ตรวจสอบว่ามีคนแจ้งแล้ว และหาตัวอย่างการใช้งานจริง